

Tugas BK 2026: Implementasi Fitur Manajemen Event (CRUD)

Deskripsi Tugas

Selamat datang di tugas Implementasi Manajemen Event! Pada tugas ini, Anda akan diminta untuk membuat sistem manajemen event lengkap dengan fitur CRUD (Create, Read, Update, Delete). Sistem ini akan memungkinkan admin untuk mengelola event konser/acara dengan sistem tiket.

Learning Outcomes

Setelah menyelesaikan tugas ini, mahasiswa diharapkan dapat:

- Memahami konsep MVC di Laravel
- Implementasi CRUD dengan Eloquent ORM
- Membuat dan menggunakan Form Request Validation
- Handle file upload dengan Storage
- Implementasi dynamic form dengan JavaScript
- Memahami Route Model Binding
- Menggunakan Query Scopes
- Implementasi soft delete logic

Prasyarat

- Laravel project sudah terinstall
- Knowledge dasar PHP dan Laravel
- Database sudah ter-setup dengan migrations:
 - `events` table
 - `tickets` table
 - `kategoris` table
 - `orders` table
 - `users` table

Bagian 1: Persiapan & Setup (10 poin)

Task 1.1: Git Branch Setup

Buat branch baru untuk fitur ini:

```
git checkout main
git pull origin main
git checkout -b feat/manajemen_event-nama
```

Task 1.2: Cek Database Structure

Pastikan migration `create_events_table.php` memiliki struktur berikut:

```
$table->id();
$table->foreignId('user_id')->constrained();
$table->foreignId('kategori_id')->constrained();
$table->string('judul');
$table->text('deskripsi');
$table->string('lokasi');
$table->string('gambar');
$table->dateTime('tanggal_waktu');
$table->timestamps();
```

Verifikasi: Jalankan `php artisan migrate:fresh --seed` untuk reset database

Bagian 2: Model Implementation (20 poin)

Task 2.1: Update Event Model

Buka `app/Models/Event.php` dan lengkapi kode berikut:

2.1.1: Define Fillable Fields

```
protected $fillable = [
    // Tambahkan field yang sesuai dengan migration
    'user_id',
    'kategori_id',
    // ... lengkapi
];
```

2.1.2: Define Relationships

Tambahkan method relationships:

- `tickets()` - hasMany relationship ke Tiket
- `kategori()` - belongsTo relationship ke Kategori
- `user()` - belongsTo relationship ke User
- `orders()` - hasMany relationship ke Order

2.1.3: Add Status Attribute

Buat accessor `getStatusAttribute()` yang mengembalikan:

- "Upcoming" jika `tanggal_waktu > now()`
- "Ongoing" jika event sedang berlangsung (dalam 3 jam terakhir)
- "Completed" jika event sudah selesai

2.1.4: Add Helper Methods

Buat method `hasSales()` yang mengembalikan `true` jika event memiliki order/penjualan.

2.1.5: Add Query Scopes

Buat scope methods:

- `scopeUpcoming($query)` - filter event yang akan datang
- `scopeOngoing($query)` - filter event yang sedang berlangsung
- `scopeCompleted($query)` - filter event yang sudah selesai

2.1.6: Add Image URL Accessor

Buat accessor `getImageUrlAttribute()` yang:

- Return URL gambar jika valid URL
- Return storage URL jika file ada di storage
- Return default image 'konser.jpg' sebagai fallback

Hint: Gunakan `filter_var($url, FILTER_VALIDATE_URL)` untuk cek valid URL

Bagian 3: Form Request Validation (20 poin)

Task 3.1: Buat EventFormRequest

```
php artisan make:request EventFormRequest
```

Task 3.2: Define Validation Rules

Buka `app/Http/Requests/EventFormRequest.php` dan lengkapi:

Rules untuk Event:

- `judul` - required, string, max 255
- `deskripsi` - required, string
- `lokasi` - required, string, max 255
- `kategori_id` - required, exists:kategoris,id
- `tanggal_waktu` - required, date, after:now
- `gambar` - nullable, image, mimes:jpg,jpeg,png, max:2048

Rules untuk Tikets (array):

- `tikets` - required, array, min:1
- `tikets.*.tipe` - required, in:reguler,premium
- `tikets.*.harga` - required, numeric, min:0
- `tikets.*.stok` - required, integer, min:0
- `tikets.*.id` - nullable, exists:tikets,id (untuk update)

Task 3.3: Define Custom Messages

Tambahkan custom error messages dalam Bahasa Indonesia:

```
return [  
    'judul.required' => 'Judul event wajib diisi.',  
    // ... lengkapi messages lainnya  
];
```

Task 3.4: Authorize Method

Pastikan hanya user dengan role `admin` yang boleh mengakses.

Bagian 4: Controller Implementation (30 poin)

Task 4.1: EventController - Index Method

Implementasi `index()` method untuk list events dengan fitur:

1. Load events dengan relationships: `kategori` dan `tickets`
2. Filter by `kategori_id` jika parameter ada
3. Search by `judul` atau `lokasi` jika parameter `search` ada
4. Sort by `tanggal_waktu` (asc/desc)
5. Paginate dengan 10 items per page

Hint: Gunakan `$request->get('sort', 'asc')` untuk default value

Task 4.2: EventController - Create & Store

`create()` Method:

- Ambil semua kategori
- Return view `pages.admin.events.create`

`store()` Method:

1. Validasi input menggunakan `EventFormRequest`
2. Handle image upload:
 - Jika ada file upload, simpan ke `storage/app/public/events`
 - Jika tidak, gunakan default `konser.jpg`
3. Create event dengan data dari form
4. Create tickets (loop through `$request->tickets`)
5. Redirect ke index dengan success message

Hint: Gunakan `$request->file('gambar')->store('events', 'public')`

Task 4.3: EventController - Edit & Update

`edit()` Method:

- Load event dan kategoris
- Load tickets dari event
- Cek `$event->hasSales()` dan pass ke view
- Return view `pages.admin.events.edit`

`update()` **Method:**

1. Validasi input
2. Jika event sudah terjual (`hasSales()`):
 - Tampilkan error jika tanggal_waktu berubah
3. Handle image update (hapus old image jika ada)
4. Update event data
5. Handle tickets:
 - Update existing tickets
 - Create new tickets
 - Delete removed tickets (hanya jika belum ada penjualan)
6. Redirect dengan success message

Task 4.4: EventController - Destroy

Implementasi `destroy()` method:

1. Cek apakah event memiliki penjualan
2. Jika ya, return dengan error message
3. Jika tidak:
 - Hapus image dari storage (jika bukan default)
 - Delete event (tickets akan terhapus via cascade)
4. Redirect dengan success message

Task 4.5: EventController - Show (Public)

Update `show()` method untuk menampilkan:

- Detail event dengan relationships
- Related events (kategori sama, tanggal > now, max 4 events)

Bagian 5: Views Implementation (40 poin)

Task 5.1: Buat Directory Views

```
mkdir -p resources/views/pages/admin/events
```

Task 5.2: Index View

Buat `resources/views/pages/admin/events/index.blade.php` dengan:

Layout:

- Header dengan title "Manajemen Event" dan button "Tambah Event"
- Success/Error alerts jika ada
- Filter form:
 - Search input (by judul/lokasi)
 - Kategori dropdown
 - Sort dropdown (asc/desc)
 - Button Filter & Reset

Table Events:

Kolom yang dibutuhkan:

- Gambar (thumbnail 64×64px)
- Judul
- Kategori
- Tanggal (format: d M Y, H:i)
- Lokasi
- Status badge (Upcoming/Ongoing/Completed)
- Actions (View, Edit, Delete dengan konfirmasi)

Pagination:

- Tampilkan pagination links
- Preserve query parameters saat pindah halaman

Hint: Gunakan `$events->appends(request()->except('page'))->links()`

Task 5.3: Create View

Buat `resources/views/pages/admin/events/create.blade.php` dengan:

Form Structure:

- Back button ke index
- Card dengan form

Fields Event (Grid 2 columns):

1. Judul Event (required)
2. Kategori dropdown (required)
3. Lokasi (required)
4. Tanggal & Waktu (datetime-local, required)
5. Gambar file input (max 2MB)
6. Image preview container (hidden by default)
7. Deskripsi textarea (required, span 2 columns)

Dynamic Ticket Form:

- "Tambah Tiket" button
- Container untuk ticket cards
- JavaScript untuk:
 - Add ticket (render card HTML)
 - Remove ticket
 - Add 1 ticket by default

Ticket Card Structure:

- Header: "Tiket #N" dengan delete button
- Grid 3 columns:
 1. Tipe Tiket (select: regular/premium)
 2. Harga (number, min 0)

3. Stok (number, min 0)

Form Layout:

Gunakan pattern berikut untuk konsistensi:

```
<div class="space-y-2">
  <label class="block">
    <span class="text-sm font-medium">Label Text</span>
    <span class="text-error">*</span>
  </label>
  <input class="input input-bordered w-full">
</div>
```

Task 5.4: Edit View

Buat `resources/views/pages/admin/events/edit.blade.php` dengan fitur tambahan:

Warning Alert:

Tampilkan warning jika `$hasSales == true` :

- Peringatan: "Event ini sudah memiliki penjualan tiket. Beberapa field mungkin tidak dapat diubah."

Form Pre-population:

- Load existing values dari `$event`
- Load existing tickets dengan JavaScript

Field Restrictions (jika hasSales):

- Tanggal & Waktu: readonly
- Warning indicator di label

Ticket Restrictions (jika hasSales):

- Ticket yang sudah terjual tidak bisa dihapus
- Badge "Sudah Terjual" pada ticket

Image Section:

- Tampilkan current image
- New image preview container
- Hint: "Kosongkan jika tidak ingin mengubah gambar"

Task 5.5: Update Public Show View

Update `resources/views/events/show.blade.php` :

Add Related Events Section:

Setelah ticket options, tambahkan:

- Section title: "Event Terkait"
- Grid 4 columns event cards
- Filter events: same category, upcoming only, max 4
- Gunakan `<x-event-card>` component

Bagian 6: Routes & Navigation (10 poin)

Task 6.1: Add Event Management Routes

Tambahkan di `routes/web.php` :

```
Route::prefix('admin')->name('admin.events.')->middleware(
    ['auth', 'verified'])->group(function () {
        Route::get('/events', [EventController::class, 'index'])->name('index');
        Route::get('/events/create', [EventController::class, 'create'])->name('create');
        Route::post('/events', [EventController::class, 'store'])->name('store');
        Route::get('/events/{event}/edit', [EventController::class, 'edit'])->name('edit');
        Route::put('/events/{event}', [EventController::class, 'update'])->name('update');
        Route::delete('/events/{event}', [EventController::class, 'destroy'])->name('destroy');
    });
```

Task 6.2: Update Admin Navigation

Buka `resources/views/layouts/admin_layouts.blade.php` dan tambahkan menu item:

```
<li>
  <a href="{{ route('admin.events.index') }}"
    class="flex items-center {{ request()->routeIs('admin.events.*') ? 'bg-blue-50'
: '' }}">
    <svg>...</svg>
    Manajemen Event
  </a>
</li>
```

Bagian 7: Testing & Verification (20 poin)

Task 7.1: Create Event Test

1. Buka halaman `/admin/events/create`
2. Isi form dengan data valid
3. Upload gambar (gunakan gambar kecil untuk testing)
4. Tambah minimal 2 tiket
5. Submit form
6. Verify: Data tersimpan di database
7. Verify: Gambar terupload ke storage
8. Verify: Redirect ke index dengan success message

Task 7.2: List & Filter Test

1. Buka halaman `/admin/events`
2. Verify: Pagination berfungsi
3. Verify: Search berfungsi (cari by judul)
4. Verify: Filter kategori berfungsi
5. Verify: Sort berfungsi (asc/desc)
6. Verify: Status badge tampil benar

Task 7.3: Update Event Test

1. Edit event yang baru dibuat
2. Verify: Form pre-populated dengan data
3. Verify: Existing tickets ter-load
4. Update data event
5. Tambah tiket baru
6. Submit form
7. Verify: Data berhasil diupdate
8. Verify: Tiket baru tersimpan

Task 7.4: Delete Event Test

1. Buat event baru
2. Hapus event tersebut
3. Verify: Event terhapus
4. Verify: Redirect dengan success message

Task 7.5: Protected Event Test

1. Buat event
 2. Simulasi ada penjualan (insert ke orders table atau cek logic)
 3. Coba hapus event
 4. Verify: Event TIDAK terhapus
 5. Verify: Error message muncul
-

Bagian 8: Bonus Challenge (20 poin ekstra)

Challenge 1: Export to Excel

Implementasi export events ke Excel dengan library seperti Laravel Excel

Challenge 2: Bulk Delete

Tambahkan fitur bulk delete dengan checkbox

Challenge 3: Image Crop

Implementasi image cropping sebelum upload

Challenge 4: Clone Event

Implementasi fitur clone/duplicate event

Challenge 5: Status History

Simpan history perubahan status event

Submission Guidelines

Yang Harus Dikumpulkan di Google Form:

1. URL Git Repository

- Push branch ke GitHub/GitLab
- Share link repository

2. URL Demo Video

- Durasi: 3-5 menit
- Show: Create, Read, Update, Delete
- Show: Dynamic ticket form
- Show: Filter & Search

3. URL Dokumentasi

- Screenshot hasil
- Kendala yang dialami
- Solusi yang ditemukan

Deadline:

- Tugas diberikan: 11 Juli 2026
 - Tugas dikumpulkan: 15 Juli 2026
-

Rubrik Penilaian

Aspek	Skor	Kriteria
Model (20 poin)	20-18	Semua fitur lengkap & benar

Aspek	Skor	Kriteria
	17-15	Mayoritas fitur lengkap
	14-12	Beberapa fitur missing
	11-0	Banyak fitur missing/error
Validation (20 poin)	20-18	Semua validasi benar & custom messages
	17-15	Validasi berfungsi, messages kurang
	14-12	Validasi basic saja
	11-0	Validasi tidak lengkap
Controller (30 poin)	30-27	Semua method berfungsi sempurna
	26-23	Method berfungsi, logic kurang
	22-18	Method basic ada, error handling kurang
	17-0	Banyak method missing/error
Views (40 poin)	40-36	UI rapi, dynamic form berfungsi
	35-31	UI baik, dynamic form ada minor issue
	30-26	UI kurang rapi, basic form
	25-0	UI berantakan/error
Routes (10 poin)	10	Semua route benar
	9-7	Route ada, grouping kurang
	6-0	Route salah/missing
Testing (20 poin)	20-18	Semua test passed
	17-15	Mayoritas test passed
	14-10	Beberapa test passed
	9-0	Testing tidak lengkap
Code Quality	+10	Code bersih, documented
Bonus	+20	Menyelesaikan challenge

Total Skor: 150 poin (+30 ekstra)

Tips Sukses

1. **Baca Laravel Docs** - Referensi resmi sangat membantu
2. **Debugging** - Gunakan `dd()` dan `logger()` untuk debug
3. **UI/UX** - Perhatikan konsistensi layout dan user experience

4. **Testing** - Test setiap fitur sebelum lanjut ke next task
 5. **Tanya** - Jangan ragu bertanya jika stuck
 6. **Commit** - Commit sering, jangan tunggu selesai semua
 7. **Fokus** - Selesaikan task per task, jangan langsung loncat
-

Resources

Documentation:

- [Laravel Documentation](#)
- [DaisyUI Components](#)
- [Tailwind CSS](#)

Useful Packages:

```
# Image manipulation
composer require intervention/image

# Excel export
composer require maatwebsite/excel

# Debugging
composer require barryvdh/laravel-debugbar
```

Quick References:

- Eloquent Relationships: <https://laravel.com/docs/11.x/eloquent-relationships>
 - Validation: <https://laravel.com/docs/11.x/validation>
 - Storage: <https://laravel.com/docs/11.x/filesystem>
 - Blade Templates: <https://laravel.com/docs/11.x/blade>
-

FAQ

Q: Boleh pakai AI helper?

A: Boleh, tapi harus paham code yang digunakan.

Q: Kalau error dan stuck gimana?

A: Cek error message, search di Google/StackOverflow, tanya mentor/teman.

Q: Deadline bisa diperpanjang?

A: Tidak, kecuali ada alasan valid dan disetujui dosen.

Q: Nilai minimum untuk lulus?

A: Minimum 75 poin (50% dari total tanpa bonus).

Good luck with your assignment!